

웹서블릿 핵심 정리와 실습

오늘의 강의 학습 요약

금일 수업은 WJSP/서블릿 환경의 기반 개념을 빠르게 복습한 뒤, 서블릿의 라이프사이클과 요청당 생성되는 `HttpServletRequest/HttpServletResponse`의 범위를 명확히 정리했습니다. 이어서 인스턴스 변수 공유로 인해 발생하는 스레드 안전성 문제를 실습으로 확인했고, 응답 처리(서블릿에서 HTML을 만들어 브라우저로 전송)와 요청 처리(폼 입력값을 서블릿에서 추출)를 GET/POST 시나리오로 다뤘습니다.

오후에는 스코프(request/session/application)의 저장, 조회, 삭제 API를 정리하고, 각 스코프의 라이프사이클과 로그인 같은 기능에서의 중요성을 연결해 설명했습니다. 마지막으로 필터의 동작 위치(서블릿 전/후), 체인 기반 실행 흐름, 매핑 방법(어노테이션/web.xml)을 실습했으며, MyBatis 기반 DB 연동을 웹 프로젝트에서 구성하는 절차와 레이어 구조(서블릿-서비스-DAO-MyBatis)를 SE 환경과 동일한 패턴으로 확장했습니다.

1. WJSP 스펙, 톰캣, 배포 구조

WJSP는 스펙(인터페이스 수준)과 구현체(벤더 서버)로 구분되며, 톰캣은 대표 구현체로 WAR 배포와 표준 디렉토리 구조를 기반으로 애플리케이션을 서비스합니다.

WJSP/서블릿은 “스펙”과 “구현”이 분리됩니다. 스펙은 인터페이스 및 규약이며, 실제 동작은 톰캣 같은 구현체가 제공합니다. 따라서 개발, 운영 환경에서는 톰캣 버전뿐 아니라 JDK 버전 호환성까지 함께 고려하여 서버 버전을 선택해야 합니다.

패키지 네임스페이스가 `javax.*` 에서 `jakarta.*` 로 바뀐 배경도 함께 정리되었습니다. 관리 주체 변화와 버전 업 과정에서 패키지명이 변경되었고, 문서/검색 결과에 두 표기가 혼재할 수 있으므로 공식 문서를 즐겨찾기해 두고 자주 확인하는 습관이 중요합니다.

톰캣에 배포(Deploy)할 때는 보통 WAR로 묶어 `webapps` 디렉토리에 배치하며, 톰캣이 자동으로 압축을 풀어 표준 구조를 만듭니다. IDE가 자동 생성해주더라도 구조 자체를 이해해야 극단적 상황(도구 없이 구성)에서 대응할 수 있습니다.

항목	핵심 내용
배포 디렉토리	<code>CATALINA_HOME/webapps</code> 가 기본 배포 위치입니다.
WAR 배포	WAR를 넣으면 톰캣이 자동으로 압축을 해제해 폴더로 전개합니다.
표준 구조	<code>WEB-INF/</code> , <code>WEB-INF/lib/</code> , <code>WEB-INF/classes/</code> , <code>web.xml</code> 구조가 핵심입니다.
주의점	<code>WEB-INF</code> 는 대문자여야 하며 표준 구조를 유지해야 합니다.

요청 URL은 `http://localhost:8080/{context}/{resource}` 형태이며, 포트 뒤에 오는 컨텍스트(context)는 물리 폴더에 대응되는 논리 이름입니다. 컨텍스트는 변경 가능하지만, URL 구성에서 “포트 뒤에 컨텍스트가 온다”는 규칙은 고정적으로 중요합니다.

핵심 키워드 `#스펙` `#톰캣` `#WAR` `#WEB-INF` `#컨텍스트`

2. 웹 컴포넌트, 서블릿 �핑, 에러코드

웹 컴포넌트는 브라우저가 요청 가능한 자원(정적/동적)이며, 서블릿은 URL �핑과 HTTP 메소드 일치가 핵심이고 불일치 시 405 오류가 발생합니다.

웹 컴포넌트는 클라이언트(웹브라우저)가 요청할 수 있는 자원이며, 정적 HTML과 동적 자원(서블릿, JSP)로 구분해서 이해합니다. 특히 서블릿은 “요청을 어떤 클래스가 받을지”를 결정하기 위해 �핑이 필수이며, 사용자가 긴 클래스 경로를 직접 입력하지 않도록 별칭(�핑 URL)을 제공합니다.

서블릿 생성 절차는 패키지 구성, `HttpServlet` 상속, `doGet/doPost` 재정의, �핑 설정 순서로 정리됩니다. `doGet` 은 GET 요청과, `doPost` 는 POST 요청과 1:1로 대응되므로 반드시 일치해야 하며, 불일치하면 405(Method Not Allowed)가 발생합니다.

구분	의미	대표 상황
404	요청한 자원을 찾을 수 없습니다.	URL/리소스 경로가 잘못되었습니다.
405	메소드가 허용되지 않습니다.	GET 요청인데 <code>doPost</code> 만 구현한 경우입니다.

�핑 방법은 어노테이션(`@WebServlet`)과 `web.xml` 두 가지가 있으며, 실무 프레임워크(예: Spring)에서는 XML 기반 설정을 보는 경우도 있어 둘 다 개념적으로 이해해야 합니다. URL 패턴은 일반적으로 슬래시(/) 기반 패턴을 사용하며, 확장자 패턴(`*.do`) 같은 방식은 과거에 많이 썼지만 현재는 거의 사용하지 않는다고 정리되었습니다.

핵심 키워드

#웹컴포넌트

#서블릿�핑

#doGet

#doPost

#405

3. 서블릿 라이프사이클과 요청당 객체 생성

서블릿 인스턴스는 최초 요청 시 1회 생성되고 `init`은 1회만 호출되며, 요청마다 `request/response` 객체가 새로 생성되어 서비스 처리 후 소멸됩니다.

서블릿은 C처럼 개발자가 `new` 로 직접 생성하는 구조가 아니라, 톰캣(서블릿 컨테이너)이 생성과 소멸을 관리합니다. 최초 요청 시 인스턴스가 없으면 생성하고 이때 `init()` 이 1회 호출됩니다. 이후 각 요청마다 `service()` 가 호출되며 내부에서 HTTP 메소드에 따라 `doGet/doPost` 가 실행됩니다.

중요한 포인트는 `HttpServletRequest` 와 `HttpServletResponse` 객체의 생존 범위입니다. 이 두 객체는 “브라우저당”이 아니라 “요청당” 생성되며, 요청-응답 사이클이 끝나면 로컬 변수처럼 제거됩니다. 즉, 같은 브라우저에서 5번 요청하면 `request/response`는 5번 생성, 소멸합니다.

요소	생성 시점	호출/생존 특징
서블릿 인스턴스	최초 요청 시(없을 때)	1회 생성 후 공유되어 재사용됩니다.
<code>init()</code>	서블릿 생성 직후	1회만 호출됩니다.
<code>doGet/doPost</code>	요청마다	매 요청마다 호출됩니다.
<code>request/response</code>	요청마다	요청-응답 사이클 동안만 존재 후 소멸합니다.

이 구조 때문에 인스턴스 변수는 여러 사용자가 공유할 수 있으며, 반대로 메소드 로컬 변수는 요청마다 새로 생성되므로 공유되지 않습니다. 이 차이가 다음 주제인 스레드 안전성과 직접 연결됩니다.

핵심 키워드 `#init` `#service` `#request` `#response` `#요청-응답 사이클`

4. 스레드 세이프/언세이프와 변수 선택

서블릿은 1개 인스턴스를 다수가 공유하므로 인스턴스 변수는 공유되어 스레드 언세이프가 되며, 로컬 변수는 요청마다 생성되어 스레드 세이프가 됩니다.

서블릿은 한 번만 생성되므로 인스턴스 변수도 한 번만 생성되고, 다수 사용자 요청이 같은 객체를 공유할 수 있습니다. 따라서 인스턴스 변수는 값이 누적되거나 다른 사용자의 변경이 섞일 수 있어 안전하지 않습니다(Thread Unsafe).

반대로 `doGet/doPost` 내 로컬 변수는 요청마다 생성되고 메소드 종료와 함께 제거되므로 다른 요청과 공유되지 않아 안전합니다(Thread Safe). 일반적인 웹 서비스는 불특정 다수가 접근하므로, 특별한 목적이 없는 한 인스턴스 변수를 상태 저장에 사용하지 않고 로컬 변수를 기본으로 선택하는 것이 핵심 원칙입니다.

변수 종류	공유 여부	스레드 안전성	웹에서의 권장
인스턴스 변수	공유됩니다.	언세이프가 될 수 있습니다.	특별한 경우가 아니면 지양합니다.
로컬 변수	공유되지 않습니다.	세이프합니다.	기본 선택으로 권장됩니다.

실습에서는 크롬(A)과 엣지(B)를 각각 사용자로 가정하여, 인스턴스 변수 카운터는 요청이 누적되어 사용자 간 공유되는 반면 로컬 변수는 매 요청 동일하게 초기화되는 현상을 확인했습니다. 이를 통해 “서블릿의 인스턴스 변수는 SE의 인스턴스 변수처럼 생각하면 안 된다”는 주의점이 강조되었습니다.

핵심 키워드

#Thread-Safe

#Thread-Unsafe

#인스턴스변수

#로컬변수

#공유

5. 응답 처리: MIME 타입, PrintWriter, 인코딩

응답 처리는 Content-Type(MIME)을 지정하고 response.getWriter()로 HTML을 출력하며, 한글 깨짐은 charset=UTF-8 설정으로 대응합니다.

응답 처리의 목적은 서버(서블릿)가 브라우저로 데이터를 보내고 브라우저가 이를 올바르게 처리하도록 만드는 것입니다. 핵심은 "어떤 데이터 타입을 보낼지"를 브라우저에 알리는 MIME(Content-Type) 지정이며, HttpServletResponse.setContentType() 로 설정합니다.

그 다음 response.getWriter() 로 PrintWriter 를 얻고, print/println 으로 HTML 문자열을 직접 작성하여 전송합니다. 이 방식은 동적 HTML 생성이 가능하지만 문자열로 태그를 작성해야 하므로 유지보수가 어렵고, 이후 JSP/템플릿 엔진을 쓰게 되는 배경으로 연결됩니다.

```
response.setContentType("text/html; charset=UTF-8");
PrintWriter out = response.getWriter();
out.println("<h1>Hello World</h1>");
out.println("<div>안녕하세요</div>");
```

MIME 타입을 text/plain 으로 주면 HTML 태그를 보내더라도 브라우저가 "텍스트"로 처리하여 태그가 그대로 보입니다. 따라서 올바른 렌더링을 위해 text/html 지정이 필수입니다.

설정	브라우저 처리	관찰 결과
text/html	HTML로 렌더링합니다.	<h1> 이 헤더로 보입니다.
text/plain	일반 문자열로 표시합니다.	태그가 그대로 출력됩니다.
charset=UTF-8	인코딩을 명시합니다.	한글 깨짐을 방지합니다.

최근 톰캣/서블릿 버전에서는 기본 처리가 개선되어 한글이 안 깨질 수 있지만, 낮은 버전 환경에서는 깨질 수 있으므로 깨짐 발생 시 ; charset=UTF-8 을 추가하는 방식으로 대응해야 합니다. 또한 API 문서에서 메소드 정의와 옵션을 직접 확인하는 습관이 강조되었습니다.

핵심 키워드

#MIME

#setContentType

#getWriter

#PrintWriter

#UTF-8

6. 요청 처리: getParameter, GET/POST, Value 배열

요청 처리는 폼 전송 데이터의 name을 키로 하여 `request.getParameter()`로 값을 문자열로 얻고, 체크박스처럼 다중 값은 `getParameterValues()`로 배열로 받습니다.

요청 처리는 사용자가 폼에 입력한 값을 서블릿에서 꺼내는 작업입니다. 폼의 `action`은 타겟(서블릿 �핑 URL)이며, `method`는 GET/POST를 결정합니다. GET은 쿼리스트링으로 URL 뒤에 붙고, POST는 바디에 담겨 URL에 노출되지 않습니다.

핵심 API는 `request.getParameter("name값")`이며, 리턴 타입은 항상 `String`입니다. 나이처럼 숫자를 입력해도 문자열로 들어오므로, 숫자 처리가 필요하다면 별도로 파싱해야 합니다. 또한 GET/POST와 `doGet/doPost`의 일치는 필수이며 불일치 시 405가 발생합니다.

전송 방식	데이터 위치	특징
GET	URL 쿼리스트링	노출되며 북마크/공유에 포함될 수 있습니다.
POST	HTTP Body	URL에 노출되지 않아 상대적으로 안전합니다.

체크박스처럼 name은 하나지만 여러 값이 전송될 수 있는 경우 `request.getParameterValues("name")`를 사용하며, `String[]`로 반환됩니다. 체크된 항목만 넘어온다는 점도 실습으로 확인했습니다.

메소드	용도	반환 타입
<code>getParameter(name)</code>	단일 값 추출	<code>String</code>
<code>getParameterValues(name)</code>	다중 값(체크박스 등) 추출	<code>String[]</code>
<code>getParameterNames()</code>	모든 파라미터 키 조회	<code>Enumeration<String></code>

또 다른 방식으로 `getParameterNames()`를 사용하면 name 목록(키)을 먼저 얻고, 반복하면서 `getParameter(key)`로 값을 조회할 수 있습니다. 반환 타입이 `Enumeration<String>`이므로 `hasMoreElements()`와 `nextElement()`로 순회하며, 이는 `Iterator`의 `hasNext()/next()`와 역할이 유사하고 메소드명만 다릅니다.

핵심 키워드 `#getParameter` `#GET` `#POST` `#getParameterValues` `#Enumeration`

7. 스코프: request/session/application 라이프사이클

스코프는 컨테이너가 관리하는 저장소의 라이프사이클이며, request는 요청-응답, session은 브라우저(타임아웃 포함), application은 톰캣 컨테이너 생존 기간과 동일합니다.

스코프는 “데이터를 어디에 저장할 것인가”의 문제이며, 단순 저장이 아니라 “언제까지 살아남아야 하는가”가 핵심입니다. 자바 변수(로컬/인스턴스/스태틱)가 라이프사이클이 다르듯, 웹에서도 컨테이너가 관리하는 저장소 3종이 서로 다른 생존 범위를 가집니다.

세 객체 모두 공통적으로 `setAttribute/getAttribute/removeAttribute` 를 제공하며, 키는 `String` , 값은 `Object` 로 어떤 타입이든 저장할 수 있습니다. 단, `getAttribute()` 는 `Object` 를 반환하므로 원래 타입으로 캐스팅이 필요합니다.

스코프	구현 API	라이프사이클(요약)
Request Scope	<code>HttpServletRequest</code>	요청-응답 사이클 동안만 유지됩니다.
Session Scope	<code>HttpSession</code>	브라우저 단위로 유지되며 타임아웃이 적용됩니다.
Application Scope	<code>ServletContext</code>	톰캣(컨테이너) 종료 전까지 유지됩니다.

세션은 `request.getSession()` 으로 얻습니다. `getSession()` 또는 `getSession(true)` 는 “없으면 생성”, `getSession(false)` 는 “없으면 null 반환”이라는 차이가 있으므로 의도에 맞게 선택해야 합니다.

세션 타임아웃은 기본 30분이며, 서버 레벨 `web.xml` 에서 기본값을 확인할 수 있습니다. 프로젝트별로는 프로젝트의 `web.xml` 에 `<session-config>` 를 설정해 서버 기본값을 재정의할 수 있으며, 단위는 “분”입니다.

이 개념은 로그인 구현에 직결됩니다. 로그인 상태를 세션에 저장해 유지하며, 타임아웃이 지나면 다시 로그인해야 하는 이유가 보안 정책(세션 만료) 때문이라는 연결까지 정리되었습니다.

핵심 키워드

`#RequestScope`

`#HttpSession`

`#ServletContext`

`#setAttribute`

`#세션 타임 아웃`

8. 스코프 실습: 값 유지 여부 검증

set 서블릿에서 3스코프에 저장 후 get 서블릿에서 조회하여, 브라우저 종료 시 session은 소멸하고 톰캣 유지 시 application은 남는 동작을 확인합니다.

실습은 "저장 서블릿(SetServlet)"과 "조회 서블릿(GetServlet)"을 분리해 스코프별 생존 차이를 눈으로 확인하는 방식으로 진행되었습니다. 저장은 `setAttribute`, 조회는 `getAttribute`를 사용하며, 조회 결과는 `Object` 이므로 문자열로 캐스팅해 출력했습니다.

브라우저를 닫으면 session 스코프는 소멸하여 null이 나오고, request 스코프는 원래 요청-응답이 끝나면 즉시 소멸하므로 이미 null입니다. 반면 톰캣이 살아 있는 동안 application 스코프는 유지되므로 값이 남습니다.

실험 조작	request	session	application
Set 호출 직후 출력	존재합니다.	존재합니다.	존재합니다.
같은 브라우저에서 Get 호출	null일 수 있습니다.	존재합니다.	존재합니다.
브라우저 종료 후 Get 호출	null입니다.	null입니다.	존재합니다.
다른 브라우저에서 Get 호출	null입니다.	null입니다.	존재합니다.

브라우저가 다르면 세션이 공유되지 않으므로, "크롬에서 로그인 후 URL을 엮지에 붙여넣으면 로그인 상태가 유지되지 않는다"는 결론으로 연결해 이해를 고정했습니다.

핵심 키워드

#getAttribute

#removeAttribute

#request스코프

#session스코프

#application스코프

9. 필터: 체인 흐름, 전/후처리, 매핑

필터는 서블릿 앞/뒤에 끼워 요청 전처리와 응답 후처리를 수행하며, `doFilter`에서 `chain.doFilter` 호출을 기준으로 전, 후 로직이 분리되고 매핑으로 적용 범위를 제어합니다.

필터는 요청이 서블릿에 도달하기 전에 먼저 실행되며, 응답이 브라우저로 가기 전에 다시 필터를 거칩니다. 즉, 서블릿 기준으로 “전처리/후처리”를 삽입하는 장치이며, 여러 개를 체인 형태로 연결할 수 있습니다.

필터 구현은 `Filter` 인터페이스를 `implements` 하고, 핵심 메소드는 `doFilter()` 입니다. 이 메소드 내부에서 `chain.doFilter(request, response)` 를 호출해야 다음 필터 또는 최종 서블릿으로 흐름이 전달됩니다.

위치	코드 위치 기준	의미
요청 필터	<code>chain.doFilter()</code> 이전	서블릿 실행 전에 수행됩니다.
응답 필터	<code>chain.doFilter()</code> 이후	서블릿 실행 후 돌아오는 시점에 수행됩니다.

적용 범위는 매핑으로 제어합니다. 특정 서블릿에만 적용하려면 필터 URL 패턴을 그 서블릿 매핑과 일치시키고, 모든 서블릿에 적용하려면 `/*` 를 사용합니다.

필터 매핑은 어노테이션(`@WebFilter`) 또는 `web.xml` 로 가능하며, `web.xml` 방식은 서블릿 매핑과 구조가 유사합니다. 실습에서는 어노테이션을 주석 처리한 뒤 `web.xml` 로 동일 동작을 재현하여 “두 설정 방식의 등가성”을 확인했습니다.

핵심 키워드

`#Filter`

`#doFilter`

`#chain.doFilter`

`#URL패턴`

`#web.xml`

10. MyBatis DB 연동: 웹 프로젝트 구성과 레이어

웹 환경의 MyBatis 연동은 SE 절차와 동일하며, JAR은 WEB-INF/lib에 넣고 설정(XML/프로퍼티), DTO, SqlSessionFactory 유틸, DAO, Service, Servlet 레이어로 연결해 목록 조회를 구현합니다.

MyBatis 연동 절차는 SE 환경에서 했던 순서와 동일하되, 라이브러리 추가 방식만 다릅니다. 웹 프로젝트에서는 빌드패스 메뉴 대신 WEB-INF/lib 에 JAR을 복사하면 클래스패스가 잡힙니다. 필요한 파일로 MyBatis JAR과 DB 커넥터(JDBC 드라이버)가 언급되었습니다.

설정 파일은 JDBC 접속 정보 프로퍼티와 MyBatis configuration.xml , 그리고 mapper.xml 두 종류가 핵심입니다. typeAlias 로 DTO 별칭을 주고, 매퍼의 namespace 와 SQL id 를 조합해 selectList("namespace.id") 로 호출합니다.

구성 요소	목적	위치/형태
JAR 2종	MyBatis + JDBC 드라이버	WEB-INF/lib 에 복사합니다.
<code>jdbc.properties</code>	URL/계정 등 4가지 접속 정보	config 패키지 등에 둡니다.
DTO	테이블 row 매핑	DeptDTO 등으로 생성합니다.
configuration/mapper XML	MyBatis 설정 및 SQL 정의	config 하위에 둡니다.

레이어 구조는 “메인 대신 서블릿이 진입점”으로 바뀌는 것만 다르고, 서비스/DAO 패턴은 그대로 유지됩니다. 서비스에서 SqlSession 을 얻고(try-finally로 close 보장), DAO에 세션을 전달하여 매퍼 SQL을 실행한 뒤 결과 리스트를 서블릿으로 반환합니다.

서블릿은 서비스 결과를 받아 응답 처리로 HTML 테이블을 생성해 출력했습니다. 이 과정에서 목록 조회(select)만 구현했으며, CRUD 나머지는 JSP 학습 이후 확장 예정이라는 흐름이 포함되었습니다.

또한 XML 파일 첫 줄의 `<?xml ... ?>` 선언은 문서의 첫 라인에 와야 하며 공백이 있으면 오류가 날 수 있다는 주의사항이 강조되었습니다.

핵심 키워드

#MyBatis

#WEB-INF/lib

#SqlSession

#Mapper

#레이어 드아키텍처